



Volume Number: 13 (1997)
Issue Number: 3
Column Tag: Programmer's Challenge

Programmer's Challenge

By Bob Boonstra, Westford, MA

Hex

This month features another round-robin tournament competition, this time with the game of Hex. You might have encountered Hex through one of Marvin Gardner's columns in Scientific American. The game is played on an NxN rhombus of hexagons, with one player attempting to occupy an unbroken chain of hexagons connecting the top and the bottom, and the other player the left with the right. Players alternate occupying hexagons, and the first to complete an unbroken chain is the winner. The prototype for the code you should write is:

```
Boolean /*legalMove*/ Hex (
    long boardSize, /* number of rows/columns in the game board */
    long oppRow,    /* row where opponent last moved, 0 .. boardSize-1 */
    long oppCol,    /* column where opponent last moved, 0 .. boardSize-1 */
    long *moveRow,  /* return your move - row, 0 .. boardSize-1 */
    long *moveCol,  /* return your move - column, 0 .. boardSize-1 */
    void *privStorage, /* preallocated storage for your use */
    Boolean newGame, /* TRUE if this is your first move in a new game */
    Boolean playFirst /* TRUE if you play first (vertically) */
);
```

For your first move, Hex will be called with newGame set to TRUE. The size of the board, a number between 8 and 64, inclusive, will be provided in boardSize. If playFirst is TRUE, your objective is to form a vertical connection across the rows of the board, and you make the first move. Otherwise you are playing second and the objective is to form a horizontal connection across the columns. Your code and the code of an opponent will alternate play. Your opponent's move will be provided in (oppRow, oppCol), which will be set to (-1, -1) if you are moving first. When your code is called, you should evaluate your opponents move, calculate your own move, and store it in (*moveRow, *moveCol). If for any reason you believe your opponent has moved into an occupied hexagon, Hex should return a legalMove value of FALSE, otherwise it should return TRUE.

A key to winning at Hex is to form "2-bridges" of hexagons that can be connected regardless of what move your opponent makes. For example, examine this 9x9 board, with the horizontal ("H") player to move:

```

0 1 2 3 4 5 6 7 8
0  . . . . . . . .
1  . . . . . V .
2  . . V . . . V .
3  . . . V . . . .
4  . . . . . V . .
5  . . V H H H V . .
6  . . . . . H . .
7  . H . V . H . .
8  . H . . . . . .
```

In this example, V can force a win. H can block one of the (row,col) positions (6,2) or (6,5), but not both. If, for example, H occupies (6,2), then V can occupy (6,5), with a guaranteed

Showcase

CAPE AND TIGHTS NOT INCLUDED

Switch from reactive to PROACTIVE support and be the superhero your users expect

WATCHMAN
MONITORING



THIS MONTH'S ISSUE



Issue 353

Code Debugging and
Spelunking

Check out the latest including:
 x86 Code Debugging and
 Spelunking ; An Introduction
 to WebGL ; Managing ESXi
 from your Mac ; Deploying
 SHA-3 ; An introduction to
 Julia ; KoolTools: BBEEdit 11.5,
 TextSoap For OS X, Honeywell
 Lyric Round Wi-Fi Thermostat,
 My Cloud EX2 Ultra ; MacTech
 Spotlight: Daniel H Steinberg

See Now



Lost Data?

We Can Save it!

- Fast
- Reliable
- Certified
- Secure

800.440.1904

www.drivesavers.com

TODAY'S DEAL

"2-bridge" connection to (7,3) via (6,4) or (7,4). A similar bridge exists if H occupies (6,5). V is therefore unstoppable from this position.

For some additional strategy, see
<<http://www.daimi.aau.dk/~tusk/pbmserve/hex/hex.faq.html>>, or
<<http://www.cs.cmu.edu/People/hde/hex/hexfaq>>. There is a "proof" that the first player can always win, but the proof is a nonconstructive proof by counterexample that does not divulge the winning strategy.

Your code will be provided with 1MB of preallocated storage pointed to by privStorage. This storage will be preinitialized to zero before your first move and will persist between moves. You should not allocate any additional dynamic storage beyond that provided by privStorage. Small amounts of static storage are permitted.

The Challenge will be scored by a tournament where each entry plays against each other entry twice for each of a number of board sizes, once playing first and once playing second. Another fair tournament schedule might be used if a large number of entries are received. For each game, the winner will earn game points for the victory, minus an amount based on the execution time used to compute the moves, as follows:

game points = 10 - (execution time in seconds)/boardSize

The loser will earn zero points and will not be penalized for the execution time expended in defeat. The player with the most total points for all games in the tournament will be the winner.

This will be a native PowerPC Challenge, using the latest CodeWarrior environment. Solutions may be coded in C, C++, or Pascal.

Three Months Ago Winner

The December Tangram Challenge was a difficult one, so difficult that no entries were submitted by the Challenge deadline. Three people (Alan Hart, Ernst Munter, and George Warner) responded with entries after I notified the Programmer's Challenge mailing list of a one-week extension, but none of those entries passed my test cases. As a result, there is no winner for the Challenge this month.

The problem was to reassemble a two-dimensional shape that has been cut into a number of smaller shapes. Each piece was to be transformed to its correct position by optionally flipping it about a vertical axis, rotating it, and translating it, in that order. A simple tangram can be created by curving a square into five right triangles of three different sizes, a smaller square, and a rhomboid. The problem statement also allowed for zero or more holes in the assembled shape, a fact which complicated the problem significantly. The submitted solutions all had problems with shapes that did not include holes, and I was unable to select one as being more correct than the rest. I have awarded 4 contest points to each of the entrants for their efforts, and selected one of them for publication.

Dave Bayer wrote to point out that the Tangram Challenge is theoretically as well as practically difficult, in that it is one of a class of problems called "NP-hard". The conjecture is that these problems cannot be solved in time that is a polynomial function of the problem size, and the problems in this class are equivalent in the sense that a polynomial time solution for one of them would provide a solution for the others. The best known problem in this class is the traveling salesperson problem.

To demonstrate that the Tangram Challenge is NP-hard, Dave transforms another NP-hard problem, the Partition Problem, into a tangram. The partition problem is to determine, given a sequence of integers i_1, i_2, i_k , whether there exists a subset whose sum is exactly half of the sum of the entire sequence. For example, given the sequence 4,4,5,7,9,11,14,14,14,18, which sums to 100, the Partition Problem would be to find a subset (e.g., 4,7,11,14,14) that sums to 50. Dave converts the sequence into a tangram with one hole by constructing pieces of a rod whose length is a number from the Partition Problem sequence, marking one half the length of the rod with a hole in the tangram. The tangram would be constructed with $3 \times N$ pieces, where N is in the sequence. Using the shorter sequence 1,2,3 to illustrate this, the tangram would be as follows:

```
XXX  XXXXXX  XXXXXXXXX          XXXXXXXXXXXXXXXXXXXX
XXX  XXXXXX  XXXXXXXXXX          XXXXXXXXXX XXXXXXXXXX
XXX  XXXXXX  XXXXXXXXXX          XXXXXXXXXXXXXXXXXXXX
```

1 2 3 assembled into 1 + 2 = 3

This construction shows that a solution to the Partition Problem would solve this tangram puzzle, and conversely, which demonstrates that the tangram problem is NP-hard. Readers interested in complexity theory can refer to *Computers and Intractability, a Guide to the Theory of NP-Completeness*, by Garey and Johnson, or to *Introduction to Automata Theory*,

ChronoSync 4.8



Synchronize & Backup Like Clockwork

Search

Community Search:

GO

MacTech Search:

GO

Software Updates via MacUpdate

Remember
when typing
felt good?



800.440.1904
www.drivesavers.com

Latest Forum Discussions

SEE ALL

iPHONE APP SHOWCASE

Price Scanner via MacPrices.net

Jobs Board

Languages, and Computation, by Hopcroft and Ullman. Thanks to Dave Bayer for these insights, and also for suggesting this month's Hex Challenge.

TOP 20 CONTESTANTS

Here are the Top Contestants for the Programmer's Challenge. The numbers below include points awarded over the 24 most recent contests, including points earned by this month's entrants.

Rank Name Points Rank Name Points

- | | |
|-------------------------|---------------------------|
| 1. Munter, Ernst 189 | 11. Cutts, Kevin 21 |
| 2. Gregg, Xan 114 | 12. Nicolle, Ludovic 21 |
| 3. Larsson, Gustav 87 | 13. Picao, Miguel Cruz 21 |
| 4. Lengyel, Eric 40 | 14. Brown, Jorg 20 |
| 5. Lewis, Peter 32 | 15. Gundrum, Eric 20 |
| 6. Boring, Randy 27 | 16. Studer, Thomas 20 |
| 7. Cooper, Greg 27 | 17. Karsh, Bill 19 |
| 8. Antoniewicz, Andy 24 | 18. Mallett, Jeff 17 |
| 9. Beith, Gary 24 | 19. Nevard, John 17 |
| 10. Kasparian, Raffi 22 | 20. Hart, Alan 14 |

There are three ways to earn points: (1) scoring in the top 5 of any Challenge, (2) being the first person to find a bug in a published winning solution or, (3) being the first person to suggest a Challenge that I use. The points you can win are:

1st place	20 points	5th place	2 points
2nd place	10 points	finding bug	2 points
3rd place	7 points	suggesting Challenge	2 points
4th place	4 points		

Here is Alan Hart's solution:

Tangrams.cp

© 1996 Alan Hart

```
/*
```

```
NOTES
```

- ```
1.Translate all the Polygons into relative vector sequences
 to do the manipulations and tests.
 Once we have a solution, convert the final positions back into
 transformations for return to the caller.

2.Holes are almost the same as pieces. Exceptions:
 - We don't need to return their transformations - They probably can't be flipped,
 - They probably can't lie along the edges of the shape.

3.The process will be recursive trial and error:

 Where pieces are used, Holes can also be included in the process,
 taking into account that they can't be flipped or deployed at the edge of the shape.

 - Put the first vertex of the first piece at the first vertex of the shape.
 - Align the first side of the piece with the first side of the shape.
 - If the piece is not entirely inside the shape, try the piece's next vertex.
 - If the piece is inside the shape modify the shape to remove the piece
 and recurse to try to place the remaining pieces.
 - If the pieces are placed successfully, return
 - When all vertices of the piece have been tried unsuccessfully, flip it and repeat
 - When both flips of the piece have been tried,
 move it to the next vertex of the shape and repeat
 - When all vertices of the shape have been tried, flip the shape and repeat

4.We need a slick procedure for testing whether one polygon lies entirely inside another. */

#include "Tangrams.h"

void SolveTangram(
```